

A Laboratory Project Report

on

LIBRARY BOOK TRACKING SYSTEM USING DOUBLY LINKED LIST

Submitted to the Data Structures Laboratory By

Mujtaba Ahmed (25261A04G3)
Neelam Varshitha (25261A04G8)
Tanusha Guraja (25261A04J1)

Course Instructors

Dr. Y. Praveen Kumar Reddy

Ms .P.Usharani / Mr. P. Vinod reddy



Department of Electronics and Communication Engineering

Mahatma Gandhi Institute of Technology (Autonomous)

(Chaitanya Bharathi P.O., Gandipeta, Hyderabad-500075)

2026

Contents

1. Introduction & motivation	1
1.1 introduction.....	1
2.2 Motivation.....	1
2. Flowcharts, Design ,& Algorithm	2
2.1 system Design.....	2
2.2 Data Structure Used.....	2
2.3 Node Design.....	2
2.4 Algorithm for Adding Book.....	3
2.5 Algorithm for Displaying a Book.....	3
2.6 Algorithm for Searching a Book.....	4
2.7 Algorithm for Issuing a Book.....	4
2.8 Algorithm for Returning a Book.....	5
2.9 Algorithm for Deleting a Book.....	5
3. Implementation Details	6
3.1 Programming Language Used.....	6
3.2 Concepts Used.....	6
3.4 Working of the Program.....	6
4 Test Plan , Result & Output	7
5. Analysis & Discussion	8
6. Conclusion & Future Work	9
7. Appendix & Code Snippets	10
Reference	

ABSTRACT

The Library Book Tracking System using Doubly Linked List is a data structure-based project developed in C language to manage library book records efficiently. Traditional manual library systems require more time for adding, searching, issuing, returning, and deleting book details. This project provides a simple menu-driven solution where each book is stored as a node in a doubly linked list.

Each node contains book details such as Book ID, title, author name, availability status, student name, issue date, and return date. The doubly linked list allows traversal in both forward and backward directions using previous and next pointers. This makes book management flexible and easy to update.

The system supports operations such as adding new books, displaying all books, searching for a book, issuing a book, returning a book, deleting a book, displaying the books, and counting total books. This project demonstrates the practical application of structures, pointers, dynamic memory allocation, and linked list operations in C.

1. Introduction & Motivation

1.1. Introduction

A library contains a large number of books, and maintaining their records manually can be difficult. Details such as book availability, issued books, return dates, and student information must be updated regularly. Manual record keeping may lead to errors, duplication, and loss of data.

To overcome these problems, this project proposes a Library Book Tracking System using a Doubly Linked List in C. A doubly linked list is a linear data structure in which each node contains data and two pointers: one pointing to the previous node and another pointing to the next node.

In this project, each book is represented as one node. The system allows the user to add, search, issue, return, delete, and display book records.

1.2. Motivation

The main motivation behind this project is to understand how data structures can be applied to real-world problems. A library system is a common example where records are frequently inserted, deleted, searched, and updated.

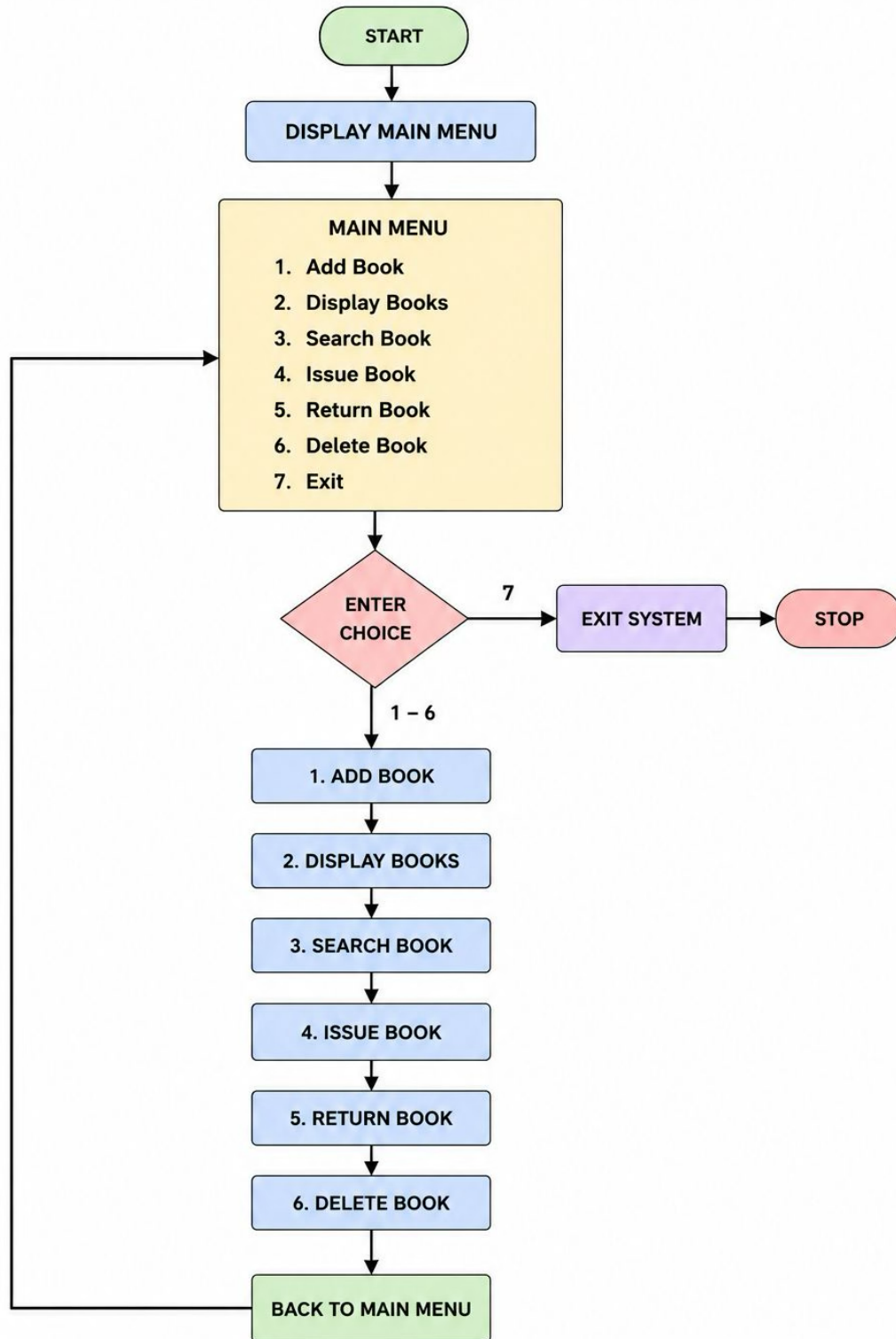
Using a doubly linked list helps us learn:

- Dynamic memory allocation
- Structure usage in C
- Pointer handling
- Forward and backward traversal
- Real-time record management

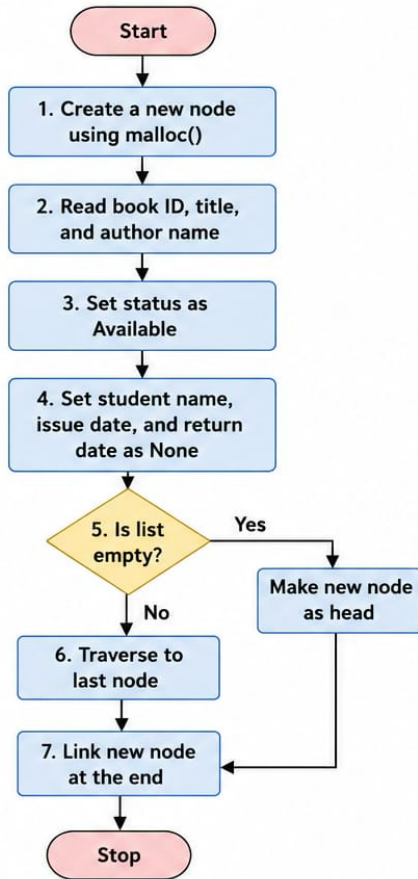
This project is useful for students because it connects theoretical data structures with practical implementation.

2. Flowcharts ,Design, & Algorithms

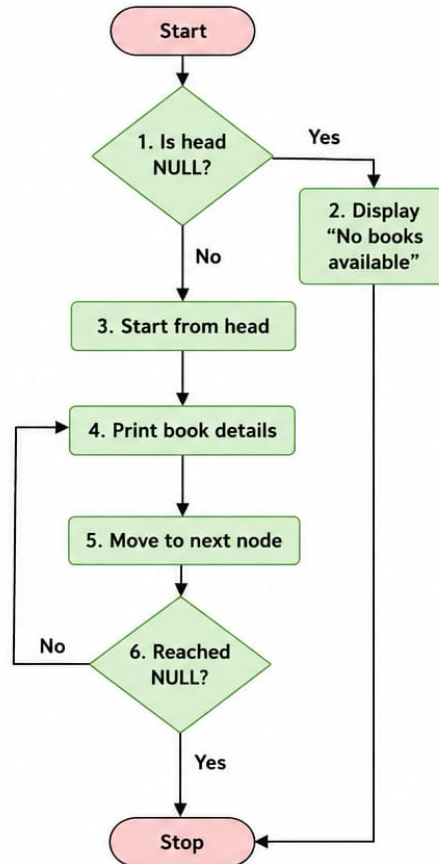
LIBRARY BOOK TRACKING SYSTEM – MAIN FLOW CHART



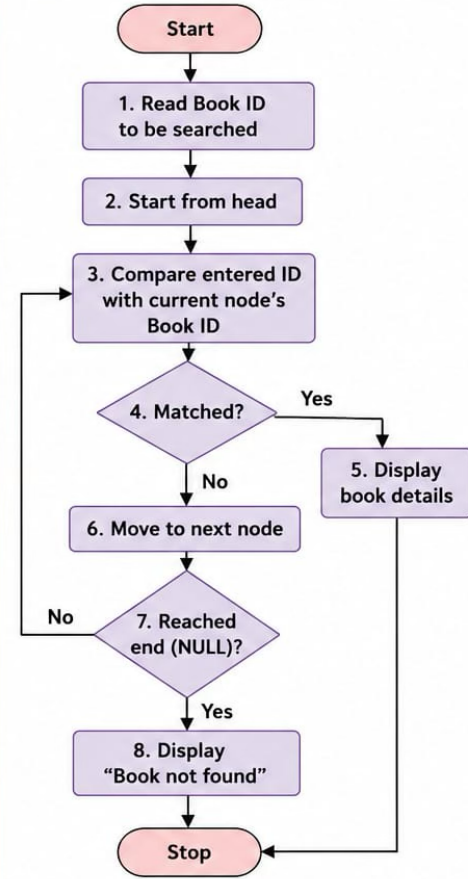
FLOWCHART FOR ADDING BOOK



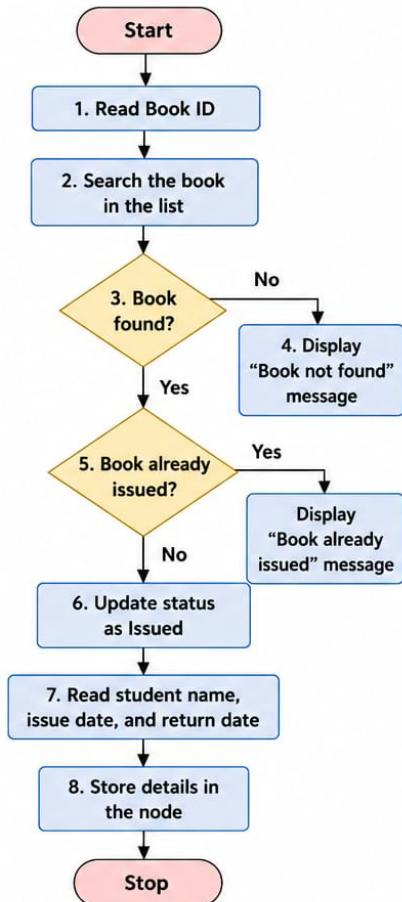
FLOWCHART FOR DISPLAYING BOOKS



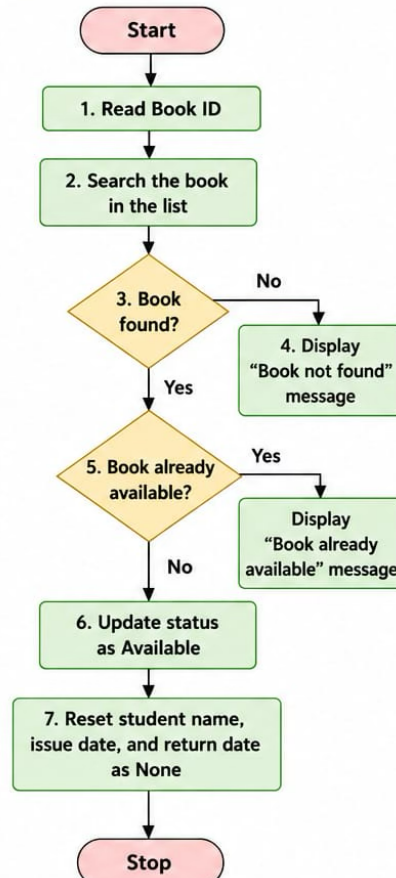
FLOWCHART FOR SEARCHING BOOK



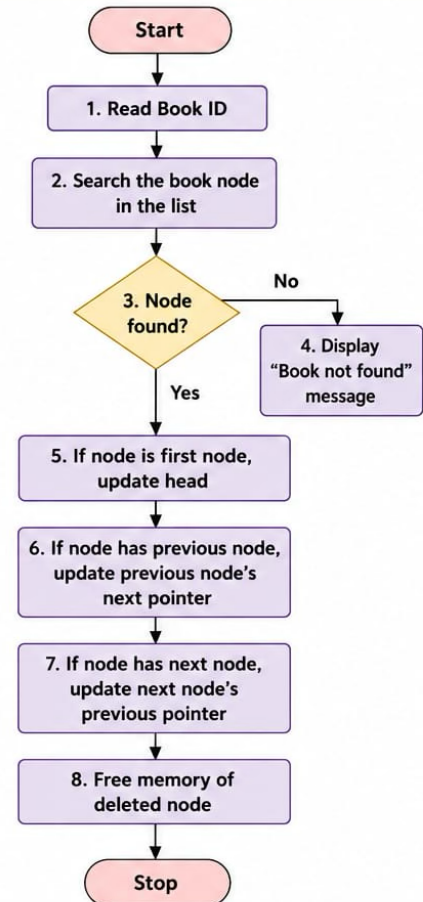
FLOWCHART FOR ISSUING A BOOK



FLOWCHART FOR RETURNING A BOOK



FLOWCHART FOR DELETING A BOOK



System Design

The system is designed as a menu-driven program. The user selects an operation from the menu, and the program performs the corresponding linked list operation.

Main operations:

1. Add Book
2. Display Books
3. Search Book
4. Issue Book
5. Return Book
6. Delete Book
7. Display Books in Reverse
8. Count Books
9. Exit

2.2. Data Structure Used

The project uses a Doubly Linked List. Each node contains:

- A previous pointer
- Data
- A next pointer

2.3. Node Design

Each node contains:

1. Book ID
2. Book Title
3. Author Name

4. Availability Status
5. Student Name
6. Issue Date
7. Return Date
8. Previous Pointer
9. Next Pointer

2.4. Algorithm for Adding a Book

- Step 1: Start
- Step 2: Create a new node using malloc()
- Step 3: Read book ID, title, and author name
- Step 4: Set status as Available
- Step 5: Set student name, issue date, and return date as None
- Step 6: If list is empty, make new node as head
- Step 7: Otherwise, traverse to last node
- Step 8: Link new node at the end
- Step 9: Stop

2.5. Algorithm for Displaying Books

- Step 1: Start
- Step 2: Check if head is NULL
- Step 3: If NULL, display “No books available”

- Step 4: Otherwise, start from head
- Step 5: Print book details
- Step 6: Move to next node
- Step 7: Repeat until NULL
- Step 8: Stop

2.6. Algorithm for Searching a Book

- Step 1: Start
- Step 2: Read Book ID to be searched
- Step 3: Start from head
- Step 4: Compare entered ID with current node's Book ID
- Step 5: If matched, display book details
- Step 6: If not matched, move to next node
- Step 7: If end is reached, display "Book not found"
- Step 8: Stop

2.7. Algorithm for Issuing a Book

- Step 1: Start
- Step 2: Read Book ID
- Step 3: Search the book in the list
- Step 4: If book is not found, display message
- Step 5: If book is already issued, display message
- Step 6: Otherwise, update status as Issued
- Step 7: Read student name, issue date, and return date

- Step 8: Store details in the node
- Step 9: Stop

2.8. Algorithm for Returning a Book

- Step 1: Start
- Step 2: Read Book ID
- Step 3: Search the book in the list
- Step 4: If book is not found, display message
- Step 5: If book is already available, display message
- Step 6: Otherwise, update status as Available
- Step 7: Reset student name, issue date, and return date as None
- Step 8: Stop

2.9. Algorithm for Deleting a Book

- Step 1: Start
- Step 2: Read Book ID
- Step 3: Search the book node
- Step 4: If node is not found, display message
- Step 5: If node is first node, update head
- Step 6: If node has previous node, update previous node's next pointer
- Step 7: If node has next node, update next node's previous pointer
- Step 8: Free memory of deleted node
- Step 9: Stop

3. Implementation Details

3.1. Programming Language Used

The project is implemented using C language.

3.2. Concepts Used

1. Structures
2. Pointers
3. Dynamic memory allocation
4. Strings
5. Functions
6. Doubly linked list
7. Menu-driven programming

3.3. Working of the Program

When the program starts, a menu is displayed. The user enters a choice. Based on the choice, the corresponding function is executed. The system keeps running until the user selects the exit option.

The head pointer stores the address of the first node. If head == NULL, it means the list is empty.

New book records are created using malloc(). Deleted records are removed using free().

4. Test Plan , Results & Output

Test Case	Input	Expected Output
Add Book	Book ID, title, author	Book added successfully
Display Books	Choose display option	All books displayed
Search Existing Book	Valid Book ID	Book details displayed
Search Invalid Book	Wrong Book ID	Book not found
Issue Available Book	Valid Book Id	Book issued successfully
Issue Already Issued Book	Same Book ID again	Book already issued
Return Issued Book	Valid Book ID	Book returned successfully
Return Available Book	Available book ID	Book already available
Delete Book	Valid Book ID	Book Deleted successfully
Count Book	Choose Count option	Count Displayed

Output

1.Add 2.Display 3.Search 4.Issue 5.Return 6.Delete 7.Reverse 8.Count 9.Exit
Enter choice: 1

Enter Book ID: 100

Enter Book Title: Winds of Fire

Enter Author Name: A.P.J.Abdul Kalam

Book added successfully

1.Add 2.Display 3.Search 4.Issue 5.Return 6.Delete 7.Reverse 8.Count 9.Exit
Enter choice: 1

Enter Book ID: 101

Enter Book Title: Think and Grow Rich

Enter Author Name: Robert Kiyosaki

Book added successfully

1.Add 2.Display 3.Search 4.Issue 5.Return 6.Delete 7.Reverse 8.Count 9.Exit

Enter choice: 1

Enter Book ID: 102

Enter Book Title: The Power of Positive Thinking

Enter Author Name: Normal Vincent Peale

Book added successfully

1.Add 2.Display 3.Search 4.Issue 5.Return 6.Delete 7.Reverse 8.Count 9.Exit

Enter choice: 1

Enter Book ID: 103

Enter Book Title: The Secret

Enter Author Name: Rhonda Byrne

Book added successfully

1.Add 2.Display 3.Search 4.Issue 5.Return 6.Delete 7.Reverse 8.Count 9.Exit

Enter choice: 1

Enter Book ID: 104

Enter Book Title: Harry Potter and the Philosopher's Stone

Enter Author Name: J.K.Rowling

Book added successfully

1.Add 2.Display 3.Search 4.Issue 5.Return 6.Delete 7.Reverse 8.Count 9.Exit

Enter choice: 1

Enter Book ID: 105

Enter Book Title: The Alchemist

Enter Author Name: Paulo Coelho

Book added successfully

1.Add 2.Display 3.Search 4.Issue 5.Return 6.Delete 7.Reverse 8.Count 9.Exit
Enter choice: 2

Library Records

Book ID: 100

Title: Winds of Fire

Author: A.P.J.Abdul Kalam

Status: Available

Student: None

Issue Date: None

Return Date: None

Book ID: 101

Title: Think and Grow Rich

Author: Robert Kiyosaki

Status: Available

Student: None

Issue Date: None

Return Date: None

Book ID: 102

Title: The Power of Positive Thinking

Author: Normal Vincent Peale

Status: Available

Student: None

Issue Date: None

Return Date: None

Book ID: 103

Title: The Secret

Author: Rhonda Byrne

Status: Available

Student: None

Issue Date: None

Return Date: None

Book ID: 104

Title: Harry Potter and the Philosopher's Stone

Author: J.K.Rowling

Status: Available

Student: None

Issue Date: None

Return Date: None

Book ID: 105

Title: The Alchemist

Author: Paulo Coelho

Status: Available

Student: None

Issue Date: None

Return Date: None

1.Add 2.Display 3.Search 4.Issue 5.Return 6.Delete 7.Reverse 8.Count 9.Exit

Enter choice: 3

Enter Book ID: 103

Book Found

Title: The Secret

Author: Rhonda Byrne

Status: Available

1.Add 2.Display 3.Search 4.Issue 5.Return 6.Delete 7.Reverse 8.Count 9.Exit

Enter choice: 4

Enter Book ID to issue: 100

Enter Student Name: surya

Enter Issue Date: 12-05-2026

Enter Return Date: 15-05-2026

Book issued

1.Add 2.Display 3.Search 4.Issue 5.Return 6.Delete 7.Reverse 8.Count 9.Exit

Enter choice: 5

Enter Book ID to return: 100

Book returned

1.Add 2.Display 3.Search 4.Issue 5.Return 6.Delete 7.Reverse 8.Count 9.Exit
Enter choice: 6

Enter Book ID to delete: 105
Deleted

1.Add 2.Display 3.Search 4.Issue 5.Return 6.Delete 7.Reverse 8.Count 9.Exit
Enter choice: 7

Reverse Order

ID: 104
Title: Harry Potter and the Philosopher's Stone

ID: 103
Title: The Secret

ID: 102
Title: The Power of Positive Thinking

ID: 101
Title: Think and Grow Rich

ID: 100
Title: Winds of Fire

1.Add 2.Display 3.Search 4.Issue 5.Return 6.Delete 7.Reverse 8.Count 9.Exit
Enter choice: 8

Total books: 5

1.Add 2.Display 3.Search 4.Issue 5.Return 6.Delete 7.Reverse 8.Count 9.Exit
Enter choice: 3

Enter Book ID: 500
Book not found

1.Add 2.Display 3.Search 4.Issue 5.Return 6.Delete 7.Reverse 8.Count 9.Exit
Enter choice: 4

Enter Book ID to issue: 200

Book not found

1.Add 2.Display 3.Search 4.Issue 5.Return 6.Delete 7.Reverse 8.Count 9.Exit

Enter choice: 5

Enter Book ID to return: 250

Book not found

1.Add 2.Display 3.Search 4.Issue 5.Return 6.Delete 7.Reverse 8.Count 9.Exit

Enter choice: 6

Enter Book ID to delete: 30

Book not found

1.Add 2.Display 3.Search 4.Issue 5.Return 6.Delete 7.Reverse 8.Count 9.Exit

Enter choice: 4

Enter Book ID to issue: 102

Enter Student Name: sai

Enter Issue Date: 11-05-2026

Enter Return Date: 15-05-2026

Book issued

1.Add 2.Display 3.Search 4.Issue 5.Return 6.Delete 7.Reverse 8.Count 9.Exit

Enter choice: 5

Enter Book ID to return: 104

Already available

1.Add 2.Display 3.Search 4.Issue 5.Return 6.Delete 7.Reverse 8.Count 9.Exit

Enter choice: 9

5. Analysis & Discussion

5.1. Time Complexity = $O(n)$ for all the functions Here, n represents the number of books in the library.

5.2. Space Complexity

The space complexity is: $O(n)$

because memory is allocated for each book node.

5.3. Advantages

1. Dynamic memory usage
2. Easy insertion and deletion
3. Forward and backward traversal
4. Simple implementation
5. Real-world application of data structures
6. Better than manual record keeping

5.4. Limitations

1. Data is lost after program termination
2. No permanent file storage
3. Search operation is linear
4. No admin login system
5. No database connection
6. No fine calculation system

5.5. Discussion

The project is suitable for understanding the concept of linked lists in a practical way. Since library records are dynamic, linked lists are more flexible than arrays. Unlike arrays, linked lists do not require fixed size memory allocation.

However, for large-scale library systems, searching through a linked list may become slow because it checks each node one by one. In future, file handling or database storage can be added to make the system more powerful.

6. Conclusion & Future Work

The project can be improved by adding:

1. File handling for permanent storage
2. Admin login system
3. Student login system
4. Fine calculation for late returns
5. Book category filtering
6. Database connectivity
7. GUI-based interface
8. Barcode or QR code scanning
9. Sorting books by title or ID
10. Faster searching using hashing or trees

7. Appendix & Code Snippets

```
#include <stdio.h> #
include <stdlib.h> #
include <string.h>

struct Book {

    int bookID; char
    title[50];
    char author[50]; char
    status[20];
    char studentName[50]; char
    issueDate[20]; char return
    Date[20];

    struct Book *prev;
    struct Book *next;
};

struct Book *newBook;

newBook = (struct Book*)malloc(sizeof(struct Book));

while (temp != NULL) {

    if (temp->bookID == id) {

        printf("Book Found"); return;
    }

    temp = temp->next;
}
```

```

if (head == NULL) {

    head = newBook;
}
else {

    struct Book *temp = head;

    while (temp->next != NULL) {

        temp = temp->next;
    }

    temp->next = newBook;
    newBook->prev = temp;
}

if (temp->prev != NULL)

    temp->prev->next = temp->next;

else

    head = temp->next;

if (temp->next != NULL)

    temp->next->prev = temp->prev;

free(temp);

```

REFERENCES

1. Reema Thareja, Data Structures Using C
2. TutorialsPoint, Data Structures in C
3. GeeksforGeeks, Doubly Linked List in C
4. OpenAI